STATESCOUT: Auditing What Must *Not* Exist — Policy-Aware, Negation-Grounded Exploration of Web User Interfaces

Team 46
Dept. of Computer Science and Engineering
Amrita Vishwa Vidyapeetham
India

Abstract — Web applications enforce authorization on the server, yet their user interfaces routinely expose what a role should never see: an Admin link rendered for guests, a debug panel shipped to production, or a destructive control two clicks from the landing page. Workflow tests miss such defects, because nobody writes a test for a button that is not supposed to exist. We present STATESCOUT, an autonomous auditor that explores a web application as a fixed role and checks every reached UI state against a role-scoped declarative policy. Our contributions are: (1) a polarity-typed policy oracle that treats forbidden-present and required-absent clauses as distinct set operations over a semantic state abstraction; (2) an error-direction asymmetry result, showing that under-reporting perception can only miss forbidden elements and can only raise false alarms on required ones, so no single perception bias is safe for both clause types, which motivates a design where deterministic DOM and accessibility-tree evidence decides existence and a vision-language model only annotates; and (3) a side-effect-aware, cyclic exploration loop with claim-before-execute semantics, implemented as a checkpointable LangGraph state machine. On a ground-truth application with six states, four cycles and four planted violations, the loop recovers the full cyclic graph with no duplicate work and reports exactly the planted violations under oracle perception. With the deterministic perception channel, precision falls to 0.29 and recall to 0.50, following the predicted asymmetry; a lexical-grounding ablation restores precision to 1.00 but not recall, isolating semantic subject grounding as the open problem.

Keywords — *access control; GUI testing; negation; policy oracle; state-space exploration; vision-language models*

I. INTRODUCTION

Broken access control heads the OWASP Top 10 [1]. Most research tools for it ask whether the *server* enforces authorization, by comparing or replaying requests under different identities [2–6]. A related class of defect lives one layer up. Consider a dashboard that renders an *Admin* link for every role. The backend correctly rejects a guest's request, so a request-level scanner reports nothing. The link itself still leaks the existence and location of privileged functionality, invites probing, and violates the organisation's least-privilege UI policy. The same applies to a debug panel left in a production build, a destructive control a guest can reach in two clicks, or the *absence* of an element the policy requires, such as a skip-to-content link, a consent banner, or a logout affordance.

Workflow-based UI testing does not catch these defects, because tests check what *should* happen and nobody writes a test for a button that should not exist. Model-inference crawlers [7–9] explore autonomously but check generic or hand-coded invariants. DOM invariants [10] can express presence and absence, but as per-page XPath predicates written in code, with no notion of role. LLM and VLM GUI agents [11–14] explore or judge screens in natural language, but they let a model issue the verdict, which is risky when the verdict is "this element is *not* there": VLMs are known to handle negation poorly [15].

This paper.. We present STATESCOUT, which reframes UI testing as *state-space auditing*. Given a seed URL, a role, and a policy of the form "a guest must never see the admin link; every page must offer a skip-to-content link", STATESCOUT explores the application autonomously, keeps the full cyclic state graph, and audits every reached state. Each finding carries the policy clause it breaks, the set operation that caught it, and human-checkable evidence.

Motivating example.. Figure 2 shows `broken-admin`, a six-page application built as ground truth. A guest lands on `index`, signs in, and reaches `dashboard`. From there the guest can open `reports` and `admin`, and every page links back. The policy, written in English and stored with its structured form, reads:

> (F1) A guest must never see the admin link. (F2) A guest must never be able to delete a user. (F3) No page may ship a debug panel. (R1) Every page must offer a skip-to-content link.

Four violations are planted. The Admin link on `dashboard` breaks F1. The debug panel on `reports` breaks F3. The Delete-user button on `admin` breaks F2. `admin` is also the only page without a skip-to-content link, which breaks R1.

Three observations follow from this example. *First*, no workflow test catches F1–F3, because each defect is extra UI, not broken UI. *Second*, R1 is an *absence*. An engine that only intersects what it sees with a forbidden list cannot express it (Section 3). *Third*, the graph is dense with cycles (log out, back to dashboard). A crawler that drops back-edges still "finds the violations" but throws away the navigation evidence that a path-sensitive policy ("reachable without signing in") would need.

Contributions.. We do not claim to be the first to crawl web applications into state graphs, to check invariants on crawled states, or to use VLMs for GUI testing; each has substantial prior art (Section II). Our contributions are a formalisation and a design that *combine* these threads for a question none of them targets: *does the UI a given role can reach conform to a declared policy of what must and must not exist?*

1. **A polarity-typed, role-scoped policy oracle** (Section A). Forbidden-present ($S \cap \mathscr{F}$) and required-absent ($\mathscr{R} \setminus S$)

are distinct set operations over a *semantic* state abstraction (roles, accessible names, tags, capabilities), not raw XPath. Polarity is part of the policy schema, and every violation is tagged with its clause type.

2. **An error-direction asymmetry result** (Proposition 2) together with its empirical confirmation (Section 2). Perception errors propagate in opposite directions for the two clause types. This argues for reporting accuracy *per polarity* and for a *negation-safe division of labour*, in which deterministic DOM/AX evidence decides existence and a VLM only annotates.
3. **A side-effect-aware cyclic exploration loop** (Section C). It uses claim-before-execute per (v,a) pair, kind-restricted replay, content-addressed actions, and normalized fingerprints, and runs as a checkpointable LangGraph state machine behind typed ports.
4. **An honest, reproducible evaluation** on a polarity-complete ground-truth application (Section IV). It separates what the loop guarantees (exact recovery with an oracle perceiver) from what perception currently delivers, and localizes the remaining gap to semantic subject grounding.

Unlike access-control scanners, StateScout does not test server-side enforcement. It audits what each role *sees*, so the two are complementary.

Organisation.. Section II reviews related work, Section III presents the formal model, the experimental materials and the implementation, Section IV reports results and discussion, and Section V concludes with future work.

II. Related Work

Table 1 summarises the positioning. No prior system combines autonomous cyclic exploration with a role-scoped oracle of both polarities. The closest neighbours are ATUSA [10] on the oracle side and BACScan [6] on the role side.

Web and GUI model inference.. GUI Ripping [8] and Crawljax [7] reverse-engineer GUIs into event-flow and state-flow graphs. WebMate [9] generates tests for Web 2.0 applications. On Android, Sapienz [24], Stoat [16], APE [17], DroidBot [18], and Humanoid [25] explore with search, stochastic models, model refinement, or learned policies, mostly with crash oracles. These models are cyclic by construction, so we do *not* claim the cyclic graph as novel. We claim only that the audit keeps it intact. State abstraction and near-duplicate detection are well studied [26]. Our fingerprint is a conventional instance, which we evaluate for stability and sensitivity but do not claim as a contribution. Doupé et al. [27] infer server state machines while tracking state-changing requests. Our claim-before-execute rule addresses the related problem of not re-firing side effects in the auditor itself.

Test oracles and invariants.. The oracle problem is surveyed by Barr et al. [28]. GUI oracles check expected state after each action [29]. ATUSA [10] checks generic and application-specific DOM invariants on every crawled state, and can express both presence and absence. Expressively, then, StateScout's oracle adds nothing that a skilled user could not hand-write as ATUSA invariants. Our delta is that polarity is part of a typed policy schema, clauses range over a semantic vocabulary that is portable across pages and markup, clauses are role-scoped, and each violation is tagged with the operation that produced it. That tagging is what enables the per-polarity analysis of Proposition 2. Property-based Android testing (Kea [19]) and accessibility checkers (MATE [20]) check properties or fixed rules during exploration, but without roles or policy polarity.

Access-control testing.. Sun et al. [2] derive per-role sitemaps statically. Li et al. [3], Noseevich and Petukhov [4], MACE [5], BACScan [6], and BACFuzz [30] detect broken access control by comparing or mutating requests across identities. These tools decide whether the *server* enforces authorization. StateScout decides whether the *UI shown to a role* conforms to a declared policy, including a required-absent half that request-level tools do not model. UI exposure is not in itself an OWASP A01 vulnerability [1], and we frame StateScout as policy conformance and least-privilege UI, complementary to request-level scanning.

LLM and VLM GUI agents and testers.. GPTDroid [11], DroidAgent [12], and AutoDroid [31] use LLMs to explore or automate mobile apps. VisionDroid [13] uses multimodal LLMs to detect non-crash functional bugs. AXNav [21] and AUITestAgent [32] turn natural-language test instructions or requirements into GUI interaction and verification, and WebTestPilot [33] infers oracles from natural-language specifications. UIGuard [22] detects dark patterns, a fixed taxonomy of *forbidden* UI. Web-agent benchmarks (WebArena [34], Mind2Web [35], OSWorld [36]) and SeeAct [14] show that grounding is the bottleneck for acting. ST-WebAgentBench [23] evaluates whether *agents* comply with policies. StateScout instead uses policy to audit the *application*. In these systems the model typically issues the verdict. StateScout withholds existence and absence judgements from the model.

Negation and compositionality in VLMs.. Alhamoud et al. [15] show that VLMs perform close to chance on negated queries. ARO [37] and CREPE [38] document bag-of-words behaviour and weak compositionality. Proposition 2 explains why this matters specifically for auditing. The damage a perception error does depends on the polarity of the clause it feeds, so negation weakness is not a uniform accuracy tax.

From natural language to specifications.. nl2spec [39] and NL2LTL [40] translate natural language into temporal logic with LLMs. StateScout's planned policy compiler targets a much simpler clause language (σ,ρ,π) with a human confirmation gate. Extending it to path-sensitive properties over the cyclic graph is where this line of work becomes directly relevant.

III. Methodology and Proposed System

A. System Overview and Problem Formulation

Figure 1 shows the architecture: a checkpointed LangGraph state machine drives a Scan→Reason→Act→Observe loop

Table 1. Positioning against the closest prior work. ✓ = stated capability, ∼ = partial or qualified, – = not a goal or absent. Columns: AE autonomous exploration; CYC cyclic state graph kept; POL declarative or NL policy; NEG "must-not-exist" oracle; REQ "required-absent" oracle; ROLE role-scoped; MM multimodal perception; SAFE side-effect safety; GT ground truth scored for exact recall. Cells for prior work reflect the cited papers as read for this draft.

Family	System	AE	CYC	POL	NEG	REQ	ROLE	MM	SAFE	GT
Model inference	Crawljax [7]	✓	✓	–	∼	∼	–	–	–	–
	ATUSA invariants [10]	✓	✓	∼	✓	✓	–	–	–	–
	WebMate [9]	✓	∼	–	–	–	∼	–	–	–
	Stoat / APE / DroidBot [16–18]	✓	✓	–	–	–	–	–	–	–
Property / a11y	Kea [19]	✓	∼	✓	∼	∼	–	–	–	∼
	MATE [20]	✓	∼	–	∼	∼	–	–	–	–
Access control	Sun et al. [2]	–	–	–	∼	–	✓	–	–	–
	Li et al. [3]	✓	∼	–	∼	–	✓	–	∼	–
	BACScan [6]	✓	∼	–	∼	–	✓	–	–	∼
LLM/VLM testing	GPTDroid / DroidAgent [11, 12]	✓	∼	–	–	–	–	–	–	–
	VisionDroid [13]	✓	∼	–	–	–	–	✓	–	∼
	AXNav [21]	–	–	✓	∼	∼	–	✓	–	–
	UIGuard [22]	–	–	∼	✓	–	–	✓	–	∼
Agent policy	ST-WebAgentBench [23]	–	–	✓	∼	∼	–	∼	∼	✓
This work	**StateScout**	✓	✓	∼	✓	✓	✓	∼	∼	∼

behind three typed ports. The remainder of this subsection formalises what the loop explores and what it checks.

1) Applications, roles, and observations. We model a web application under test (AUT), as seen by a fixed role r, as a labelled transition system $M_r = (Q, A, \delta, q_0)$ over concrete UI configurations Q, actions A, and a partial transition function $\delta : Q \times A \rightharpoonup Q$. The auditor never sees Q directly. A *capture* $\kappa(q) = (u, d, t, s)$ bundles the URL, serialized DOM, accessibility (AX) tree, and an optional screenshot. StateScout pins one role per run; multi-role coverage is several runs, and comparing roles is a reporting-layer concern (Section A).

Definition 1 (State abstraction)**.** *A* fingerprint *is* $\phi = H \circ N$*, where N normalizes a capture by removing volatile content (session and CSRF tokens, nonces, long numeric timestamps, generated hex identifiers, per-session CDP node ids, transient focus), and H is SHA-256. Two configurations are* the same abstract state *iff* $\phi(\kappa(q)) = \phi(\kappa(q'))$.

Normalization has two failure modes. If it is too sensitive, the model explodes into one node per page load. If it is too coarse, a security-relevant change (for example, a destructive button that appears) folds into an existing state and is never audited. Section 4 measures both.

2) The exploration graph is cyclic.

Definition 2 (Exploration graph)**.** *The* exploration graph *is a directed multigraph $G = (V, E)$ with $V \subseteq \operatorname{img}\phi$ and $E \subseteq V \times A \times V$, where $(v, a, v') \in E$ records that executing a in v was observed to lead to v'. An edge is a* back-edge *if v' was already discovered when it was added.*

Model-inference crawlers often treat revisits as redundant and drop them, leaving a tree or DAG. We treat back-edges as *evidence*. "Log out returns to login" and "Admin returns to Dashboard" are facts about the AUT's navigation structure, and a path-sensitive audit can only reason over them if they are kept. StateScout therefore never prunes an edge. Loop prevention lives entirely in a *claim set* $C \subseteq V \times A$.

Definition 3 (Claim-before-execute)**.** *Action a is executed from v only if $(v, a) \notin C$, and (v, a) is inserted into C* before *a is sent to the browser. The edge (v, a, v') is persisted only* after *a succeeds.*

Proposition 1 (Termination and at-most-once per pair)**.** *Suppose $|V| \le n$, which is enforced by a state cap, and each state enables at most k actions. Then exploration performs at most nk* frontier *executions and terminates. No claimed pair (v, a) is executed as a frontier action twice, including across crash and resume.*

Proof sketch. Every frontier execution inserts a distinct pair into C first, and $|C| \le nk$. On resume, a pair that is in C but has no edge is treated as done and never re-fired. Execution strictly follows insertion, so a crash cannot cause a second execution of the same pair. □

Most AUTs are not idempotent: re-submitting a form changes the application being audited. Reaching a queued action with a single browser, however, means *replaying* its path from the seed, and replay re-fires the actions on that path. Proposition 1 is therefore a guarantee about *pairs*, not about *effects*. StateScout narrows the gap by kind: only $\mathsf{ReplaySafe} = \{\texttt{navigate}, \texttt{click}, \texttt{back}\}$ may be re-fired. A candidate whose path passes through a `fill`, `select`, or `submit` is recorded as skipped, with a reason, and its subtree is reported as unexplored instead of being reached by quietly re-submitting a form. The residual assumption is that *clicks are navigational*. A click that mutates state on a replayed path (for example, a one-click "Delete") *is* re-fired. We treat this as an open limitation (Section A), not a solved problem. Action identity is *content-addressed*: $id(a) = H(role \,\|\, name \,\|\, selector)$, not a DOM index. Replay therefore survives DOM reordering, and the same "Log

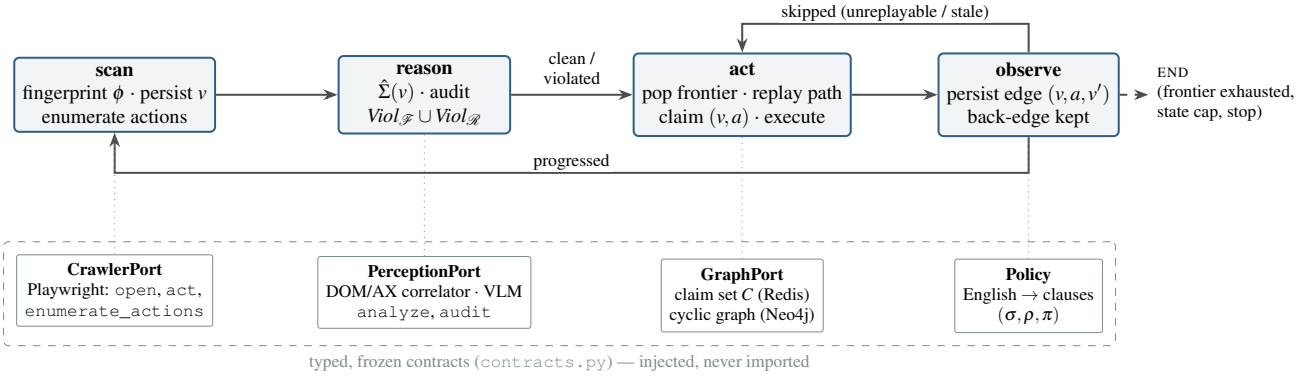


Figure 1. STATESCOUT architecture. A checkpointed LangGraph state machine of four nodes and two conditional edges drives the Scan→Reason→Act→Observe loop. All I/O goes through three typed ports, so every component can be replaced by a deterministic fake for testing and by the live implementation in production without changing the loop.

out" control keeps one identity everywhere, while deduplication stays scoped to (v,a).

3) Policies as two set operations. A policy is written in English by a QA engineer and compiled into machine-checkable clauses $c = (\sigma_c, \rho_c, \pi_c)$: a *subject* σ_c (a UI element kind or capability), a role scope ρ_c (empty means all roles), and a polarity $\pi_c \in \{\text{must_exist}, \text{must_not_exist}\}$. This splits the policy into forbidden clauses $\mathscr{F}$ and required clauses $\mathscr{R}$. Perception maps each state to a set of grounded subjects $\Sigma(v)$ (element tags, accessible names, and capabilities). For a role r, let $\mathscr{F}_r$ and $\mathscr{R}_r$ be the clauses whose scope admits r.

Definition 4 (Negation-aware audit)**.**

$$Viol_{\mathscr{F}}(v) = \{\, c \in \mathscr{F}_r : \sigma_c \in \Sigma(v) \,\} \quad \textit{(forbidden present)} \tag{1}$$

$$Viol_{\mathscr{R}}(v) = \{\, c \in \mathscr{R}_r : \sigma_c \notin \Sigma(v) \,\} \quad \textit{(required absent)} \tag{2}$$

The audit of v is $Viol_{\mathscr{F}}(v) \cup Viol_{\mathscr{R}}(v)$, and each violation is tagged with the operation that produced it.

Equation (1) is an intersection ($S \cap \mathscr{F}$) and Eq. (2) is a difference ($\mathscr{R} \setminus S$). No single intersection can express both, so an intersection-only oracle is structurally unable to report a missing element (Section 3).

4) Error-direction asymmetry under imperfect perception. In a real system, $\Sigma(v)$ is replaced by a perception estimate $\hat{\Sigma}(v)$ from a DOM/AX correlator or a vision-language model. The two clause types react to perception errors in *opposite* directions.

Proposition 2 (Error-direction asymmetry)**.** *Let $\widehat{Viol}$ denote the audit computed on $\hat{\Sigma}$.*

1. *If perception* under-reports *($\hat{\Sigma}(v) \subseteq \Sigma(v)$), then $\widehat{Viol}_{\mathscr{F}}(v) \subseteq Viol_{\mathscr{F}}(v)$ and $\widehat{Viol}_{\mathscr{R}}(v) \supseteq Viol_{\mathscr{R}}(v)$. Forbidden clauses can only be missed, and required clauses can only raise false alarms.*
2. *If perception* over-reports *($\hat{\Sigma}(v) \supseteq \Sigma(v)$, for example by hallucinating a subject), both inclusions reverse.*

Proof. Equation (1) is monotone in Σ and Eq. (2) is antitone in Σ. □

Corollary 1. *No uniform perception bias is safe for both clause types. A detector tuned to avoid false alarms on forbidden clauses (conservative, under-reporting) creates false alarms on required clauses, and the reverse also holds.*

This matters for multimodal perception. Vision-language models answer negated queries close to chance [15], and their compositional understanding is known to be shallow [37, 38]. Those studies use natural images, not UI screenshots, so we cite them as motivation rather than as a measurement of UI perception. Whatever a perceiver's error rate, Proposition 2 shows that a single yes/no "is σ on the screen?" query has a polarity-dependent risk profile. This motivates the design choices in Section 5: evidence-carrying verdicts, a deterministic DOM/AX channel that cross-checks the VLM, and (proposed) a three-valued verdict that abstains instead of guessing when the two channels disagree. Section 2 observes exactly the asymmetry predicted by Proposition 2.

B. Materials, Dataset and Tools

Tools.. The system is written in Python (about 5.8k lines across five modules) with a Playwright crawler, a LangGraph orchestrator, Redis for the claim set, Neo4j for the cyclic graph, a FastAPI reporting service, and a VS Code extension front-end. The vision-language channel uses Gemini 2.5 Flash behind a rate-limited provider interface; it is implemented but not evaluated here.

Dataset: subject application.. `broken-admin` is a static six-page application (index, login, register, dashboard, reports, admin) with four cycles and four planted violations, browsed as `guest` (Fig. 2). The policy has three forbidden clauses (admin link, delete-user, debug panel) and one required clause (skip-to-content link). V-01 is an Admin link rendered for every role; the backend rejects the request, but the link itself is the exposure. V-02 is a debug panel left in the production build. V-03 is a Delete-user control two clicks from the seed. V-04 is a missing skip-to-content link, which is an *absence*.

Ground-truth format.. Each test application ships three machine-readable files next to its HTML: `app.json` (pages and controls), `policy.json` (the English policy

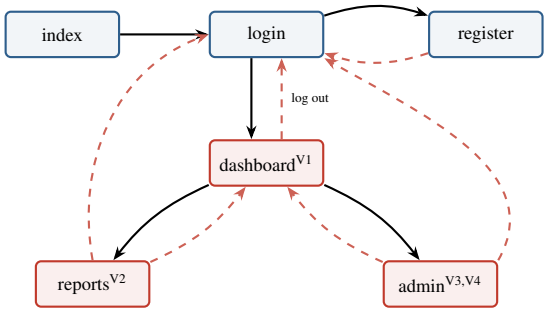


Figure 2. Exploration graph recovered for `broken-admin` (role `guest`): 6 states, 11 edges, of which 6 are back-edges (dashed). A tree- or DAG-based crawler would drop every dashed edge. Superscripts mark planted violations.

and its structured form), and `violations.json` (every planted defect with its page and clause). Evaluation asserts *exact* set equality between found and planted violations. A miss is a regression, and an undocumented finding is a false positive. A consistency test fails if the HTML and `app.json` drift apart.

Reproducibility.. All numbers are produced by `docs/demo_crawl.py` (RQ1) and `docs/report/experiments/run_experiments.py` (RQ2–RQ4). The raw output is in `docs/report/data/experiments.json`. Every run is deterministic.

C. Algorithm and Implementation Details

Figure 1 shows the architecture. The implementation consists of about 5.8k lines of Python across five modules owned by four sub-teams, plus a FastAPI reporting service and a VS Code extension front-end.

1) Orchestration. The explorer performs breadth-first search over (v, a) pairs. A plain-Python reference loop, kept as a *parity oracle*, is ported to a LangGraph `StateGraph` with four nodes (`scan`, `reason`, `act`, `observe`) and two conditional edges. The whole exploration state (frontier, claimed pairs, discovery order, skips) is a single checkpointed `TypedDict`, so a crashed run can resume instead of restarting. Limits such as depth, state cap, and perception rate are configuration rather than state, so a resumed run cannot silently run under different rules. A separate conditional edge routes a skipped action back to `act` and not through `scan`. Otherwise an unchanged state would be re-audited and a phantom entry added to the discovery order, and the discovery order is the evidence for cycles. The same test suite runs against both implementations.

2) Capture and actions. A Playwright crawler captures DOM, the CDP full AX tree, and a screenshot. It derives candidate actions from actionable AX roles (link, button, tab, menuitem, ...) and gives each one a content-addressed identifier. Navigation failures and action failures raise distinct exception types, so the orchestrator can retry the former with backoff and mark-and-skip the latter without type-sniffing.

3) State abstraction and persistence. ϕ removes known volatile attributes (`nonce`, `data-session`, `data-csrf`, ...), hex-like generated ids, prefixed tokens (`sess_`, `csrf_`, `tok_`), 10+-digit timestamps, whitespace, and volatile CDP keys (`nodeId`, `backendDOMNodeId`, `parentId`, `childIds`, focus). It then hashes URL, DOM, and canonical AX JSON together. The claim set lives in Redis. States, edges (including back-edges), and violations are stored in Neo4j, where the cyclic structure can be queried directly (for example, "all paths from the seed to a violating state").

4) Perception: grounding subjects in the UI. Perception turns a capture into a *Semantic UI Map*: elements with role, accessible name, tags, selector, visibility and enabled state, plus the capabilities the state exposes (`delete-user`, `export-data`, ...). The baseline channel is deterministic and fully reproducible. It correlates DOM elements and AX nodes by (i) browser-provided identifiers, (ii) role plus accessible name, and (iii) name alone when unique. Ambiguous matches are never guessed. DOM-only elements are kept on purpose, because an element hidden from the AX tree can still be an exposure. A vision-language model channel (Gemini 2.5 Flash, behind a rate-limited provider interface) reads the screenshot and AX tree. It supplies judgements the DOM cannot make, such as a styled `<div>` with no ARIA role that *looks like* a delete button. By design it may only *annotate* elements the deterministic channel already found, adding tags and capabilities. It may never add or remove an element, so existence and absence (the polarity that VLMs are known to handle poorly [15]) are decided by evidence that cannot be hallucinated. In the current codebase this channel is implemented but not yet wired into the orchestrator, and it is not evaluated in this paper. A run without screenshots is labelled a smoke test, not an audit.

5) Negation engine. The audit implements Eqs. (1) and (2) deterministically. Clause applicability is checked against the run's role. Subjects are matched first against capabilities, which are stronger evidence, and then against element tags and accessible names. There is no substring matching, because an over-eager matcher is exactly the over-reporting bias of Proposition 2. Every violation carries its clause type, a severity, a plain-language rationale, and *evidence* (selector, text, screenshot) that a human can check. The policy compiler that turns English sentences into clauses is *in progress*. By design it reuses the perception port's text channel, so there is one model client and one rate limit, and it includes a confirmation gate on which the QA engineer approves the parse before the crawl. In this draft, policies are supplied in structured form next to their English source (`policy.json`).

IV. Results and Discussion

We ask four research questions:

RQ1 Does the exploration loop reach every state, keep the graph cyclic, avoid duplicate work, and report exactly the planted violations?

RQ2 How does the perception channel's subject grounding affect audit accuracy, and does the error follow Proposition 2?

RQ3 Is the required-absent operation (Eq. (2)) necessary,

Table 2. RQ1: end-to-end crawl of `broken-admin` (oracle perception).

Metric	Value
Termination reason	frontier exhausted
States discovered / expected	6 / 6
Edges persisted (of which back-edges)	11 (6)
(v,a) executed / distinct claimed	11 / 11
Duplicate (v,a) executions	0
Path replays / replayed steps	9 / 21
Actions skipped	0
Violations found / planted	4 / 4
Precision / recall	1.00 / 1.00

or is an intersection-only oracle enough?

RQ4 Is the fingerprint ϕ stable under volatile content and still sensitive to real UI changes?

1) RQ1: End-to-end exploration. We run the LangGraph loop on `broken-admin` with two stand-ins: an *oracle perception* that reads the planted `data-tag` annotations, and a *scripted crawler* generated from the application's `app.json`, not a live browser. This isolates exploration and audit logic from perception and browser behaviour. RQ1 therefore establishes the correctness of the loop and the audit algebra, not end-to-end accuracy. RQ2 measures what happens when the oracle is removed. Table 2 shows the results. The run ends by *frontier exhaustion*, not by a cap. It finds all six states and records all 11 transitions, including 6 back-edges. The 11 executed (v,a) pairs are exactly the 11 distinct claimed pairs, so there is no duplicate work. The four reported violations match the ground truth exactly, with no extras. Breadth with a single browser costs 9 path replays (21 replayed steps), a 1.9× overhead over the 11 executed actions, which is reported rather than hidden. The full suite (545 tests passing, 4 skipped only because Neo4j and Redis were not running) checks these properties against both loop implementations.

2) RQ2: Perception grounding and error direction. We keep the negation engine fixed and swap the perception channel on all six pages (Table 3). The *deterministic DOM channel* builds the Semantic UI Map from the HTML with rule-based tagging and no annotations. It reaches precision 0.29 and recall 0.50. The error pattern matches Proposition 2. The channel *under-reports*: it grounds the admin link as the generic tag `admin` and the debug section as `debug`, not as the policy subjects `admin-link` and `debug-panel`. Both missed violations are therefore **forbidden-present false negatives**. The same under-reporting turns the required clause into **five required-absent false positives**: the link text "Skip to content" is never matched to the subject `skip-to-content`, so the clause fires on every page that does carry the link. The channel produces no forbidden false positives and no required false negatives, as the proposition predicts.

The ablation *+lexical grounding* normalizes hyphens and underscores in the matcher. It removes all five false positives (precision 0.29→1.00) but leaves recall at 0.50. The remaining misses are *semantic*, not lexical: deciding that a link labelled "Admin" is an `admin-link`, or that a `<section>` whose heading reads "Debug" is a `debug-panel`, requires the kind of judgement a VLM or a learned grounding model has to provide. This result pinpoints the research problem that remains: *subject grounding* between the vocabulary of a policy and the vocabulary of a rendered interface.

Table 3. RQ2/RQ3: audit accuracy on `broken-admin` by perception channel and audit operator. $\mathrm{FP}_{\mathscr{F}}/\mathrm{FN}_{\mathscr{F}}$ and $\mathrm{FP}_{\mathscr{R}}/\mathrm{FN}_{\mathscr{R}}$ split errors by clause type.

Configuration	P	R	F1	$\mathrm{FN}_{\mathscr{F}}$	$\mathrm{FP}_{\mathscr{F}}$	$\mathrm{FN}_{\mathscr{R}}$	$\mathrm{FP}_{\mathscr{R}}$
Oracle, $\mathscr{F}\cup\mathscr{R}$	1.00	1.00	1.00	0	0	0	0
Oracle, $\mathscr{F}$ only (RQ3)	1.00	0.75	0.86	0	0	1	–
DOM channel	0.29	0.50	0.36	2	0	0	5
DOM + lexical grounding	1.00	0.50	0.67	2	0	0	0

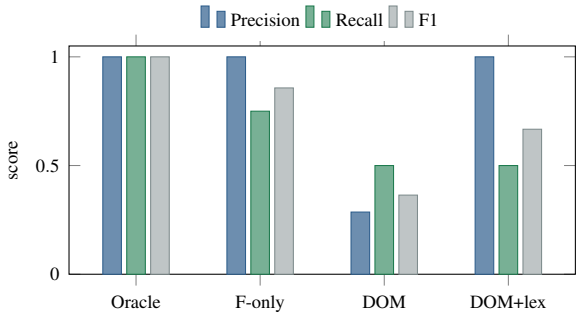


Figure 3. Audit accuracy by configuration. Lexical grounding fixes precision (required-absent false alarms) but not recall (semantic grounding of forbidden subjects).

3) RQ3: Is "required absent" necessary?. Dropping $\mathscr{R}$ and auditing with the intersection $S\cap\mathscr{F}$ alone keeps precision at 1.00 but loses V-04 (recall 0.75). No setting of the forbidden clauses can recover it, because an absence is not an element that can intersect anything. This is the empirical counterpart of the structural argument in Section A. Accessibility obligations, consent banners, and logout affordances are all required-absent properties.

4) RQ4: Fingerprint stability and sensitivity. We re-render each of the six pages 50 times and inject fresh per-request noise into every capture: `data-session`/`data-csrf` tokens, a generated hex `id`, a `data-request-id`, a 13-digit timestamp, and a CDP-shaped AX tree with new `nodeId`s. A naive hash of URL, DOM, and AX produces **300 distinct states** for 6 logical pages (one per load). ϕ produces exactly **6**, one per page (Table 4). Normalization does not over-merge: deleting the *Delete user* button from the admin page changes its fingerprint, so a security-relevant difference still yields a distinct state.

5) Threats to validity. External: there is one small, static, hand-built application with four planted defects. The results show the mechanism works, not that it generalises, and Section A outlines a benchmark to address this. *Construct:* RQ1 uses an oracle perception by design. RQ2 evaluates the deterministic channel on HTML without a live AX tree, and the VLM channel is implemented but not evaluated

Table 4. RQ4: distinct state ids over 50 noisy captures per page.

Page	naive hash	ϕ (normalized)
`/index.html`	50	1
`/pages/login.html`	50	1
`/pages/register.html`	50	1
`/pages/dashboard.html`	50	1
`/pages/reports.html`	50	1
`/pages/admin.html`	50	1
Total (6 logical states)	300	6
Real change (Delete button removed) detected?		yes

here. The RQ4 noise model covers known volatile patterns and not, for example, rotating advertisements or A/B variants. *Internal:* the lexical-grounding ablation patches the matcher's normalizer in-process only. It measures an effect and is not a shipped change.

A. Discussion and Research Agenda

Subject grounding is the crux.. RQ2 shows that once exploration and the audit algebra are correct, accuracy depends on one mapping: from a policy *subject* ("admin link") to the elements and capabilities of a rendered state. Lexical normalization closes the easy half. The semantic half, deciding what an element *is* and what it *lets the user do*, is where a VLM earns its cost, and also where negation weakness and hallucination come in. Proposition 2 suggests evaluating grounding *separately per polarity*. A single accuracy number hides whether a model fails in the direction that misses forbidden exposures or in the direction that raises false alarms on required ones.

Proposed: three-valued, cross-checked verdicts.. Given the asymmetry, we propose extending the audit from $\{\mathsf{clean}, \mathsf{violated}\}$ to a Kleene-style $\{\mathsf{clean}, \mathsf{violated}, \mathsf{indeterminate}\}$. The verdict is indeterminate when the deterministic DOM/AX channel and the VLM channel disagree on $\sigma \in \Sigma(v)$. An indeterminate forbidden clause is escalated to a human with evidence from both channels and is never silently cleared. This turns the corollary of Proposition 2 into a routing rule instead of a tuning problem.

Scope predicates beyond roles.. Clauses are currently scoped by role only. The test application shows why this is not enough. "Every *signed-in* page must offer a logout control" would fire on the login page, where a logout control is correctly absent. The fix is a page-scope predicate on clauses (for example, over URL patterns or state properties). In the limit, that means path-sensitive properties over the cyclic graph ("no state reachable from *login* without authenticating exposes `delete-user`"). Such properties are exactly why back-edges must be kept.

Replay safety beyond action kinds.. Proposition 1 guarantees at-most-once per claimed pair, not per effect. Restricting replay to $\{\texttt{navigate}, \texttt{click}, \texttt{back}\}$ excludes form fills and submissions, but it assumes that clicks are navigational. A one-click destructive control on a replay path violates that assumption. Candidate refinements are (i) classifying controls as mutating from their accessible name and capability tags before they are made replayable, (ii) snapshot-and-restore of the AUT between replays where the deployment allows it, and (iii) multi-browser exploration that parks one browser per frontier state, which reduces replay at the cost of memory.

Cross-role differential auditing.. Because a role-gated element changes the DOM, and therefore the fingerprint, running StateScout once per role yields one graph per role. Aligning these graphs by URL and action identity gives a *differential* view ("what does a guest see at this URL that only an admin should?"). That view needs no hand-written forbidden list, and we plan to add it as a reporting-layer analysis.

Towards a benchmark.. To move beyond one application, we propose NegUI, a benchmark of seeded applications. Each ships `app.json`, `policy.json`, and `violations.json`, and violations are stratified by clause type (forbidden-present vs. required-absent), by grounding difficulty (lexical, semantic, visual-only), and by depth. Planned seeds include the existing RBAC, feature-flag, and consent-banner demos in the repository. Scoring is exact-set agreement, reported per polarity.

V. Conclusion and Future Work

StateScout audits what a role can *see*, where workflow tests check what a user can *do*. Formalising the policy oracle as two typed set operations makes absence a first-class violation. It also exposes a structural fact: perception errors propagate in opposite directions for forbidden and required clauses, so no single perception bias is safe for both. On a polarity-complete ground-truth application, the exploration loop and audit algebra are exact when perception is an oracle. The deterministic perception channel shows the predicted asymmetry, and a lexical-grounding ablation isolates semantic subject grounding as the open problem.

The next steps follow directly: (i) run the pipeline end to end with the live Playwright crawler and the deterministic and VLM perception channels; (ii) evaluate presence and absence judgements separately per polarity on a probe set of hard cases (unlabelled `<div>` controls, hidden and icon-only elements); (iii) add page-scope predicates and three-valued verdicts; (iv) narrow replay safety beyond action kinds; and (v) release NegUI with a Crawljax-plus-invariants baseline.

Artifact.. The code, the ground-truth applications and the scripts that produce every number are in the project repository (`docs/report/experiments/`).

References

[1] OWASP Foundation, "A01:2021 – broken access control." OWASP Top 10:2021, 2021.

[2] F. Sun, L. Xu, and Z. Su, "Static detection of access control vulnerabilities in web applications," in *Proceedings of the 20th USENIX Security Symposium*, 2011.

[3] X. Li, X. Si, and Y. Xue, "Automated black-box detection of access control vulnerabilities in web applications," in *Proceedings of the 4th ACM Conference on Data and Application Security and Privacy (CODASPY)*, pp. 49–60, 2014.

[4] G. Noseevich and A. Petukhov, "Detecting insufficient access control in web applications," in *First SysSec Workshop (SysSec)*, pp. 11–18, 2011.

[5] M. Monshizadeh, P. Naldurg, and V. N. Venkatakrishnan, "MACE: Detecting privilege escalation vulnerabilities in web applications," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 690–701, 2014.

[6] F. Liu, Y. Zhang, E. Li, W. Meng, Y. Shi, Q. Wang, C. Wang, Z. Lin, and M. Yang, "BACScan: Automatic black-box detection of broken-access-control vulnerabilities in web applications," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2025.

[7] A. Mesbah, A. van Deursen, and S. Lenselink, "Crawling Ajax-based web applications through dynamic analysis of user interface state changes," *ACM Transactions on the Web*, vol. 6, no. 1, pp. 3:1–3:30, 2012.

[8] A. M. Memon, I. Banerjee, and A. Nagarajan, "GUI ripping: Reverse engineering of graphical user interfaces for testing," in *Proceedings of the 10th Working Conference on Reverse Engineering (WCRE)*, pp. 260–269, 2003.

[9] V. Dallmeier, M. Burger, T. Orth, and A. Zeller, "WebMate: Generating test cases for web 2.0," in *Software Quality. Increasing Value in Software and Systems Development (SWQD 2013)*, Springer, 2013.

[10] A. Mesbah, A. van Deursen, and D. Roest, "Invariant-based automatic testing of modern web applications," *IEEE Transactions on Software Engineering*, vol. 38, no. 1, pp. 35–53, 2012.

[11] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang, and Q. Wang, "Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, 2024.

[12] J. Yoon, R. Feldt, and S. Yoo, "Intent-driven mobile GUI testing with autonomous large language model agents," in *Proceedings of the IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2024. arXiv preprint title: "Autonomous Large Language Model Agents Enabling Intent-Driven Mobile GUI Testing" (arXiv:2311.08649).

[13] Z. Liu, C. Li, C. Chen, J. Wang, M. Chen, B. Wu, Y. Wang, J. Hu, and Q. Wang, "Vision-driven automated mobile GUI testing via multimodal large language model," 2024. Later arXiv versions retitled "Seeing is Believing: Vision-driven Non-crash Functional Bug Detection for Mobile Apps" (tool renamed Trident), whose author list adds Mengzhuo Chen; the v1 title shown here listed 8 authors without him. Cite the version you read.

[14] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su, "GPT-4V(ision) is a generalist web agent, if grounded," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, vol. 235 of *PMLR*, 2024.

[15] K. Alhamoud, S. Alshammari, Y. Tian, G. Li, P. H. S. Torr, Y. Kim, and M. Ghassemi, "Vision-language models do not understand negation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[16] T. Su, G. Meng, Y. Chen, K. Wu, W. Yang, Y. Yao, G. Pu, Y. Liu, and Z. Su, "Guided, stochastic model-based GUI testing of Android apps," in *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, 2017.

[17] T. Gu, C. Sun, X. Ma, C. Cao, C. Xu, Y. Yao, Q. Zhang, J. Lu, and Z. Su, "Practical GUI testing of Android applications via model abstraction and refinement," in *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pp. 269–280, 2019.

[18] Y. Li, Z. Yang, Y. Guo, and X. Chen, "DroidBot: A lightweight UI-guided test input generator for Android," in *Proceedings of the 39th International Conference on Software Engineering Companion (ICSE-C)*, pp. 23–26, 2017.

[19] Y. Xiong, T. Su, J. Wang, J. Sun, G. Pu, and Z. Su, "General and practical property-based testing for Android apps," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024.

[20] M. M. Eler, J. M. Rojas, Y. Ge, and G. Fraser, "Automated accessibility testing of mobile apps," in *IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*, pp. 116–126, 2018.

[21] M. Taeb, A. Swearngin, E. Schoop, R. Cheng, Y. Jiang, and J. Nichols, "AXNav: Replaying accessibility tests from natural language," in *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI)*, 2024.

[22] J. Chen, J. Sun, S. Feng, Z. Xing, Q. Lu, X. Xu, and C. Chen, "Unveiling the tricks: Automated detection of dark patterns in mobile applications," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[23] I. Levy, B. Wiesel, S. Marreed, A. Oved, A. Yaeli, N. Mashkif, and S. Shlomov, "ST-WebAgentBench: A benchmark for evaluating safety and trustworthiness in web agents," in *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2410.06703.

[24] K. Mao, M. Harman, and Y. Jia, "Sapienz: Multi-objective automated testing for Android applications," in *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA)*, pp. 94–105, 2016.

[25] Y. Li, Z. Yang, Y. Guo, and X. Chen, "Humanoid: A deep learning-based approach to automated black-box Android app testing," in *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2019. Listed in the ASE 2019 Demonstrations track on the conference site.

[26] R. Yandrapally, A. Stocco, and A. Mesbah, "Near-duplicate detection in web app model inference," in *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*, 2020.

[27] A. Doupé, L. Cavedon, C. Kruegel, and G. Vigna, "Enemy of the state: A state-aware black-box web vulnerability scanner," in *Proceedings of the 21st USENIX Security Symposium*, 2012.

[28] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.

[29] A. M. Memon, M. E. Pollack, and M. L. Soffa, "Automated test oracles for GUIs," in *Proceedings of the 8th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pp. 30–39, 2000.

[30] I. P. A. Dharmaadi, M. Alhanahnah, V.-T. Pham, F. Mohsen, and F. Turkmen, "BACFuzz: Exposing the silence on broken access control vulnerabilities in web applications," 2025. Preprint, under review at time of retrieval.

[31] H. Wen, Y. Li, G. Liu, S. Zhao, T. Yu, T. J.-J. Li, S. Jiang, Y. Liu, Y. Zhang, and Y. Liu, "AutoDroid: LLM-powered task automation in Android," in *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom)*, 2024.

[32] Y. Hu, X. Wang, Y. Wang, Y. Zhang, S. Guo, C. Chen, X. Wang, and Y. Zhou, "AUITestAgent: Automatic requirements oriented GUI function testing," 2024.

[33] X. Teoh, Y. Lin, D.-M. Nguyen, R. Ren, W. Zhang, and J. S. Dong, "WebTestPilot: Agentic end-to-end web testing against natural language specification by inferring oracles with symbolized GUI elements," 2026.

[34] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, "WebArena: A realistic web environment for building autonomous agents," in *International Conference on Learning Representations (ICLR)*, 2024.

[35] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, "Mind2Web: Towards a generalist agent for the web," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[36] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei, Y. Liu, Y. Xu, S. Zhou, S. Savarese, C. Xiong, V. Zhong, and T. Yu, "OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[37] M. Yuksekgonul, F. Bianchi, P. Kalluri, D. Jurafsky, and J. Zou, "When and why vision-language models behave like bags-of-words, and what to do about it?," in *International Conference on Learning Representations (ICLR)*, 2023.

[38] Z. Ma, J. Hong, M. O. Gul, M. Gandhi, I. Gao, and R. Krishna, "CREPE: Can vision-language foundation models reason compositionally?," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10910–10921, 2023.

[39] M. Cosler, C. Hahn, D. Mendoza, F. Schmitt, and C. Trippel, "nl2spec: Interactively translating unstructured natural language to temporal logics with large language models," in *Computer Aided Verification (CAV)*, Springer, 2023.

[40] F. Fuggitti and T. Chakraborti, "NL2LTL – a Python package for converting natural language (NL) instructions to linear temporal logic (LTL) formulas," in *Proceedings of the AAAI Conference on Artificial Intelligence (System Demonstrations)*, vol. 37, 2023.